# Bài 1.2 – Python cốt lõi (2): hàm, class, đọc/ghi file

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *Câu chuyện*

## 🎯 Sau bài này bạn sẽ

- Viết **hàm** để không phải copy-paste code.
- Hiểu **class** đủ để đọc code thư viện ML (vì mọi mô hình trong Scikit-learn, PyTorch đều là class).
- Đọc/ghi file văn bản, CSV, JSON.
- Xử lý lỗi bằng `try/except` để chương trình không "chết" giữa chừng.

**Chia buổi:**

- **Buổi 1:** Mục 1 → 4 (hàm, lambda).
- **Buổi 2:** Mục 5 → 9 (class, file, xử lý lỗi).

---

## 1. Câu chuyện: Chị Hoa và 47 chỗ copy-paste

Chị Hoa làm báo cáo doanh thu cho chuỗi 12 quán phở. Mỗi tuần, chị copy đoạn code tính tiền có giảm giá vào 47 chỗ khác nhau.

Một hôm, sếp đổi luật: "*Giảm giá tối đa 30%, không được hơn.*"

Chị Hoa phải sửa 47 chỗ. Chị sửa sót 3 chỗ. Báo cáo sai. Sếp không vui.

Nếu ngay từ đầu chị viết **một hàm** `tinh_tien()` và gọi nó 47 lần, chị chỉ cần sửa **một chỗ**.

> **Nguyên tắc DRY – Don't Repeat Yourself (Đừng lặp lại chính mình):** khi thấy mình copy-paste lần thứ 3, hãy viết hàm.

---

## 2. Hàm (Function)

### 2.1. Trực giác

Hàm giống **máy xay sinh tố**: bỏ nguyên liệu vào (tham số), bấm nút, nhận ra ly sinh tố (giá trị trả về). Bạn không cần biết bên trong máy hoạt động thế nào mỗi lần dùng.

```
giá, số lượng, giảm giá  ──►  [ tinh_tien() ]  ──►  số tiền phải trả
    (tham số / input)           (thân hàm)          (return / output)
```

### 2.2. Code

```
def tinh_tien(gia, so_luong, giam_gia=0.0):
    """Tính tiền phải trả. giam_gia là tỉ lệ, ví dụ 0.1 = giảm 10%."""    #
docstring: mô tả hàm
```

```
    tong = gia * so_luong                # biến tong chỉ tồn tại BÊN TRONG hàm
    return tong * (1 - giam_gia)         # return: trả kết quả ra ngoài

print(tinh_tien(55000, 2))               # không truyền giam_gia → dùng mặc định
0.0
print(tinh_tien(55000, 2, giam_gia=0.1))   # truyền theo tên: rõ ràng, dễ đọc
```

**Kết quả mẫu:**

```
110000.0
99000.0
```

Vì sao ra `110000.0` mà không phải `110000`? Vì nhân với `(1 - 0.0)` là số thực, kết quả thành số thực.

### 2.3. Các khái niệm

- **Parameter (Tham số):** tên biến trong định nghĩa hàm, như `gia`, `so_luong`.
- **Default value (Giá trị mặc định):** `giam_gia=0.0`, dùng khi người gọi không truyền.
- **Keyword argument (Truyền theo tên):** `giam_gia=0.1`. Thư viện ML dùng kiểu này rất nhiều, ví dụ `train_test_split(X, y, test_size=0.2)` (sẽ học kỹ ở bài 1.11).

### 2.4. Trả về nhiều giá trị

```
def thong_ke(ds_so):
    return min(ds_so), max(ds_so), sum(ds_so) / len(ds_so)   # trả về 3 giá trị
một lúc

# "Mở gói" 3 giá trị vào 3 biến
nho_nhat, lon_nhat, trung_binh = thong_ke([40000, 55000, 20000, 45000])
print(nho_nhat, lon_nhat, trung_binh)
```

**Kết quả mẫu:**

```
20000 55000 40000.0
```

Bạn sẽ gặp mẫu này mãi: `X_train, X_test, y_train, y_test = train_test_split(...)`.

---

## 3. Lambda: hàm một dòng không tên

- **Định nghĩa:** Một hàm ngắn viết trong 1 dòng, thường dùng một lần rồi bỏ.
- **Ví dụ đời thực:** Như tờ giấy nhớ viết vội "sắp theo giá", so với một cuốn sổ tay (hàm `def` đầy đủ).

```
lambda m: m["gia"]
       │    └─ trả về cái gì
       └─ nhận vào cái gì
```

```
ds_mon = [
    {"ten": "Phở bò",  "gia": 55000, "danh_gia": 4.7},
    {"ten": "Bún chả", "gia": 45000, "danh_gia": 4.5},
    {"ten": "Bánh mì", "gia": 20000, "danh_gia": 4.8},
]

# Sắp xếp theo giá tăng dần: key cho biết "so sánh dựa trên cái gì"
xep_theo_gia = sorted(ds_mon, key=lambda m: m["gia"])
print([m["ten"] for m in xep_theo_gia])

# Sắp xếp theo đánh giá giảm dần
xep_theo_sao = sorted(ds_mon, key=lambda m: m["danh_gia"], reverse=True)
print([m["ten"] for m in xep_theo_sao])
```

**Kết quả mẫu:**

```
['Bánh mì', 'Bún chả', 'Phở bò']
['Bánh mì', 'Phở bò', 'Bún chả']
```

---

# 4. Bài tập nhanh Buổi 1

1. Sửa hàm `tinh_tien` theo luật mới của sếp chị Hoa: nếu `giam_gia` lớn hơn 0.3 thì chỉ giảm 0.3. Kiểm tra: `tinh_tien(100000, 1, giam_gia=0.5)` phải ra `70000.0`.
2. Viết hàm `phan_loai_gia(gia)` trả về `"Rẻ"` (dưới 30.000), `"Vừa"` (30.000 đến dưới 60.000), `"Đắt"` (từ 60.000).

## Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

```
def tinh_tien(gia, so_luong, giam_gia=0.0):
    giam_gia = min(giam_gia, 0.3)        # không cho vượt 30%
    return gia * so_luong * (1 - giam_gia)
print(tinh_tien(100000, 1, giam_gia=0.5))   # 70000.0

def phan_loai_gia(gia):
    if gia < 30000:
        return "Rẻ"
    elif gia < 60000:
        return "Vừa"
```

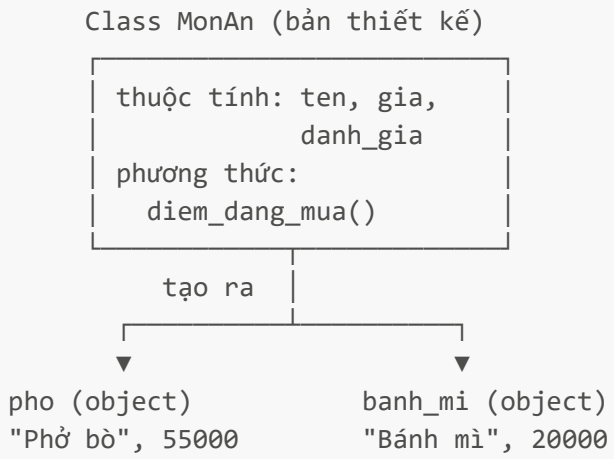
```
    return "Đắt"
print([phan_loai_gia(g) for g in [20000, 45000, 250000]])   # ['Rẻ', 'Vừa', 'Đắt']
```

---

# 5. Class (Lớp) – vì sao người học ML phải biết?

## 5.1. Trực giác

- **Class** là **bản thiết kế**. Ví dụ: bản vẽ "món ăn" ghi rằng món nào cũng có tên, giá, đánh giá.
- **Object (Đối tượng)** là **sản phẩm làm ra từ bản thiết kế**. Ví dụ: tô phở cụ thể, ổ bánh mì cụ thể.
- Một class gồm:
  - **Attribute (Thuộc tính):** dữ liệu nó mang theo (tên, giá).
  - **Method (Phương thức):** việc nó làm được (tính điểm đáng mua).




## 5.2. Code

```
class MonAn:
    def __init__(self, ten, gia, danh_gia):    # __init__: chạy tự động khi tạo
object
        self.ten = ten                         # self = "chính object này"
        self.gia = gia                         # lưu dữ liệu vào object
        self.danh_gia = danh_gia

    def diem_dang_mua(self):                   # phương thức: hàm nằm trong class
        # số sao trên mỗi 10.000đ: càng cao càng "đáng đồng tiền"
        return round(self.danh_gia / (self.gia / 10000), 2)

    def __repr__(self):                        # quy định cách object hiện ra khi
print
        return f"MonAn({self.ten}, {self.gia:,}đ, {self.danh_gia}★)"

pho = MonAn("Phở bò", 55000, 4.7)              # tạo object từ class
banh_mi = MonAn("Bánh mì", 20000, 4.8)
print(pho)
```

```
print(pho.diem_dang_mua(), banh_mi.diem_dang_mua())   # gọi phương thức bằng dấu
chấm
```

**Kết quả mẫu:**

```
MonAn(Phở bò, 55,000đ, 4.7★)
0.85 2.4
```

Kiểm tra bằng tay: Phở bò 4,7 ÷ (55.000 ÷ 10.000) = 4,7 ÷ 5,5 ≈ **0,85**. Bánh mì 4,8 ÷ 2 = **2,4**. Bánh mì đáng đồng tiền hơn.

### 5.3. Mẫu class quan trọng nhất trong ML: `fit` và `predict`

Mọi mô hình trong Scikit-learn đều theo khuôn:

```
mo_hinh = TenMoHinh()        # 1. tạo object
mo_hinh.fit(du_lieu)         # 2. học từ dữ liệu → lưu "kiến thức" vào thuộc tính
mo_hinh.predict(du_lieu_moi) # 3. dùng kiến thức đã học để dự đoán
```

Hãy tự viết một "mô hình" đồ chơi theo đúng khuôn đó:

```
class BoGoiYDonGian:
    """Mô hình đồ chơi: món rẻ hơn giá trung bình là 'Rẻ', ngược lại là 'Đắt'."""

    def __init__(self):
        self.gia_trung_binh = None              # chưa học thì chưa biết

    def fit(self, ds_gia):
        self.gia_trung_binh = sum(ds_gia) / len(ds_gia)   # "học": ghi nhớ giá
trung bình
        return self                             # trả về chính nó (thói quen của
sklearn)

    def predict(self, gia_moi):
        # dùng điều đã học để phân loại giá mới
        return ["Rẻ" if g < self.gia_trung_binh else "Đắt" for g in gia_moi]

bo = BoGoiYDonGian()
bo.fit([55000, 45000, 20000, 40000])
print("Giá trung bình học được:", bo.gia_trung_binh)
print(bo.predict([30000, 70000]))
```

**Kết quả mẫu:**

```
Giá trung bình học được: 40000.0
['Rẻ', 'Đắt']
```

Bạn vừa tự viết một "mô hình ML" tí hon! Khi gặp `LinearRegression().fit(X, y)` ở bài 2.2, bạn sẽ thấy quen thuộc.

---

# 6. Đọc và ghi file

## 6.1. File văn bản

```
# Ghi file. "w" = write (ghi đè). encoding="utf-8" để không lỗi tiếng Việt
with open("thuc_don.txt", "w", encoding="utf-8") as f:
    f.write("Phở bò,55000\n")         # \n = xuống dòng
    f.write("Bún chả,45000\n")
    f.write("Bánh mì,20000\n")

# Đọc file. "r" = read
ds = []
with open("thuc_don.txt", "r", encoding="utf-8") as f:
    for dong in f:                              # đọc từng dòng
        ten, gia = dong.strip().split(",")     # strip() bỏ \n, split(",") cắt
theo dấu phẩy
        ds.append({"ten": ten, "gia": int(gia)})   # int() đổi chuỗi "55000" thành
số
print(ds)
```

**Kết quả mẫu:**

```
[{'ten': 'Phở bò', 'gia': 55000}, {'ten': 'Bún chả', 'gia': 45000}, {'ten': 'Bánh
mì', 'gia': 20000}]
```

`with` **để làm gì?** Tự động đóng file khi xong, kể cả khi có lỗi. Giống cửa tự đóng.

| Chế độ | Ý nghĩa |
|---|---|
| `"r"` | Đọc |
| `"w"` | Ghi đè (xóa nội dung cũ!) |
| `"a"` | Ghi thêm vào cuối (append) |

File bạn vừa đọc chính là **CSV – Comma-Separated Values (giá trị ngăn cách bằng dấu phẩy)**, định dạng dataset phổ biến nhất. Ở bài 1.4, Pandas đọc CSV chỉ bằng 1 dòng: `pd.read_csv(...)`.

## 6.2. File JSON

- **JSON – JavaScript Object Notation:** định dạng lưu dữ liệu trông giống hệt list/dict của Python.
- **Ví dụ đời thực:** dữ liệu API trả về, file cấu hình mô hình, nhãn dữ liệu ảnh (sẽ gặp ở bài CV3.4).

```
import json

# Ghi list dict ra file JSON. ensure_ascii=False để giữ nguyên tiếng Việt
with open("mon_an.json", "w", encoding="utf-8") as f:
    json.dump(ds, f, ensure_ascii=False, indent=2)    # indent=2: thụt lề cho dễ
đọc

# Đọc lại: nhận về đúng list dict ban đầu
with open("mon_an.json", "r", encoding="utf-8") as f:
    doc_lai = json.load(f)
print(doc_lai[0]["ten"], type(doc_lai))
```

**Kết quả mẫu:**

```
Phở bò <class 'list'>
```

---

# 7. Xử lý lỗi với try/except

## 7.1. Vấn đề

Dữ liệu thật **luôn bẩn**. Cột giá có thể chứa `"45k"`, `"liên hệ"`, ô trống... Nếu gọi `int("45k")`, chương trình dừng ngay với lỗi `ValueError`. Với file 1 triệu dòng, chết ở dòng 999.999 thì rất đau.

## 7.2. Code

```
def doc_gia(chuoi):
    try:                                # thử làm việc có thể lỗi
        return int(chuoi)
    except ValueError:                  # nếu lỗi đúng loại ValueError thì làm
việc này
        print(f"  Không đọc được giá: '{chuoi}'")
        return None                     # None = "không có giá trị"

print([doc_gia(s) for s in ["55000", "45k", "20000"]])
```

**Kết quả mẫu:**

```
  Không đọc được giá: '45k'
[55000, None, 20000]
```

Chương trình chạy hết, đánh dấu chỗ lỗi bằng `None` để xử lý sau. Bài 1.6 sẽ dạy cách xử lý những ô "không có giá trị" này.

> ⚠️ Đừng viết `except:` trống không. Nó nuốt **mọi** lỗi, kể cả lỗi do bạn gõ sai tên biến, khiến bạn không biết code hỏng ở đâu.

---

## 8. Bài tập Buổi 2

1. Thêm vào class `MonAn` phương thức `giam_gia(ti_le)` để giảm giá món đó. Ví dụ `pho.giam_gia(0.1)` thì `pho.gia` thành `49500.0`.
2. Viết hàm `doc_thuc_don(ten_file)` đọc file `thuc_don.txt` ở mục 6.1 và trả về **list các object MonAn**, với `danh_gia` mặc định là 4.0.
3. **Tìm lỗi:** Đoạn code dưới đây chạy không lỗi nhưng file cuối cùng chỉ có 1 dòng. Vì sao?

```
for mon in ["Phở bò", "Bún chả", "Bánh mì"]:
    with open("log.txt", "w", encoding="utf-8") as f:
        f.write(mon + "\n")
```

### Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

```
# 1. Thêm vào trong class MonAn:
    def giam_gia(self, ti_le):
        self.gia = self.gia * (1 - ti_le)

# 2.
def doc_thuc_don(ten_file):
    ket_qua = []
    with open(ten_file, "r", encoding="utf-8") as f:
        for dong in f:
            ten, gia = dong.strip().split(",")
            ket_qua.append(MonAn(ten, int(gia), 4.0))
    return ket_qua
print(doc_thuc_don("thuc_don.txt"))
# [MonAn(Phở bò, 55,000đ, 4.0★), MonAn(Bún chả, 45,000đ, 4.0★), MonAn(Bánh mì,
20,000đ, 4.0★)]
```

3. Chế độ `"w"` **xóa file cũ** mỗi lần mở. Vòng lặp mở file 3 lần nên chỉ còn dòng cuối "Bánh mì". Sửa: dùng `"a"`, hoặc đưa `with open(...)` ra **ngoài** vòng lặp.

---

## 9. Flashcard

| Mặt trước | Mặt sau |
|---|---|
| Khi nào nên viết hàm? | Khi một đoạn code bị lặp lại (nguyên tắc DRY). |

| Mặt trước | Mặt sau |
|---|---|
| `return` làm gì? | Trả kết quả ra ngoài hàm và kết thúc hàm. |
| Lambda là gì? | Hàm 1 dòng không tên, ví dụ `lambda m: m["gia"]`. |
| Class vs Object? | Class là bản thiết kế. Object là sản phẩm làm từ bản thiết kế. |
| `self` là gì? | Chính object đang được dùng. |
| Hai phương thức "quốc dân" của mô hình ML? | `fit()` để học, `predict()` để dự đoán. |
| Vì sao dùng `with open(...)`? | Tự đóng file kể cả khi có lỗi. |
| `"w"` vs `"a"`? | `"w"` ghi đè, `"a"` ghi thêm vào cuối. |
| Vì sao cần `try/except`? | Dữ liệu thật bẩn; không để một dòng lỗi làm dừng cả chương trình. |

**Bài tiếp theo:** 1.3 – NumPy: mảng và phép toán vector hóa.